⚙️ EduTrack — Backend Development Document

Academic Management Platform

VTU Internship 2026 — Team 15

1. Spring Initializr Setup

The backend project was initialized using **Spring Initializr** (https://start.spring.io) with the following configuration:

SETTING	VALUE
Project	Maven
Language	Java
Spring Boot Version	3.2.3
Group	com.edutrack
Artifact	edutrack-backend
Packaging	JAR
Java Version	21

The generated project was imported into Eclipse IDE using: **File → Import → Maven → Existing Maven Projects**

2. Package Structure

The project follows a standard layered architecture with the following packages under `com.edutrack`:

```
com.edutrack/
├── EduTrackApplication.java      # Main entry point
├── config/                       # Configuration classes
│   ├── SecurityConfig.java       # Spring Security + CORS config
```

```
│   └── AdminSeeder.java          # Default admin auto-creation
├── controller/                   # REST API endpoints
│   ├── AuthController.java
│   ├── AppControllers.java       # Student, Teacher, Course, Attendance,
Marks
│   ├── OtpController.java
│   ├── PasswordResetController.java
│   ├── BulkImportController.java
│   ├── CourseMaterialController.java
│   ├── SubmissionController.java
│   ├── SystemSettingsController.java
│   └── HealthController.java
├── service/                      # Business logic
│   ├── AuthService.java
│   ├── OtpService.java
│   └── EmailService.java
├── repository/                   # Database access (Spring Data JPA)
│   ├── AdminRepository.java
│   ├── StudentRepository.java
│   ├── TeacherRepository.java
│   ├── CourseRepository.java
│   ├── AttendanceRepository.java
│   ├── AssignmentRepository.java
│   ├── MarksRepository.java
│   ├── SubmissionRepository.java
│   ├── CourseMaterialRepository.java
│   ├── OtpRepository.java
│   └── SystemSettingsRepository.java
└── entity/                       # JPA entity classes (DB tables)
    ├── Admin.java
    ├── Student.java
    ├── Teacher.java
    ├── Course.java
    ├── Attendance.java
    ├── Assignment.java
    ├── Marks.java
    ├── Submission.java
    ├── CourseMaterial.java
    ├── OtpVerification.java
    └── SystemSettings.java
```

LAYER	PACKAGE	RESPONSIBILITY
Controller	`controller`	Handles HTTP requests and responses
Service	`service`	Contains business logic

LAYER	PACKAGE	RESPONSIBILITY
Repository	`repository`	Database queries using Spring Data JPA
Entity	`entity`	Maps Java classes to database tables
Config	`config`	Security, CORS, and seeding configuration

3. Application Properties

The `application.properties` file contains all configuration for the backend. A template is provided as `application.properties.template` in the repository.

```
# Server
server.port=8080

# MySQL Database
spring.datasource.url=jdbc:mysql://localhost:3306/edutrack?
useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=YOUR_DB_PASSWORD
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

# App name
spring.application.name=edutrack-backend

# Email SMTP (Gmail)
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=YOUR_GMAIL_ADDRESS
spring.mail.password=YOUR_GMAIL_APP_PASSWORD
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

3a. Creating Gmail App Password

To enable OTP email sending, a Gmail App Password is required:

1. Go to **myaccount.google.com**
2. Click **Security**
3. Enable **2-Step Verification** (must be ON)
4. Search for **App Passwords**
5. Select app: **Mail**, device: **Other** → type "EduTrack"
6. Click **Generate**
7. Copy the **16-character password** shown
8. Paste it as `spring.mail.password` in `application.properties`

3b. Password Length

The Gmail App Password is exactly **16 characters** (e.g. `abcd efgh ijkl mnop` — spaces are ignored).

3c. Server Port

The backend runs on port **8080** by default:

```
server.port=8080
```

4. Dependencies (pom.xml)

All dependencies are managed through **Maven** in `pom.xml`.

4.1 Core Dependencies

DEPENDENCY	PURPOSE
`spring-boot-starter-web`	REST API support (Spring MVC, Tomcat)
`spring-boot-starter-data-jpa`	ORM with Hibernate and Spring Data
`mysql-connector-j`	MySQL JDBC driver
`spring-boot-starter-security`	Authentication and CORS configuration
`spring-boot-starter-validation`	Bean validation (@NotNull, @Email, etc.)
`spring-boot-starter-mail`	JavaMailSender for OTP emails via Gmail SMTP
`lombok`	Reduces boilerplate (@Data, @Getter, @Setter)
`commons-csv`	CSV parsing for bulk import feature

DEPENDENCY	PURPOSE
`spring-boot-starter-test`	JUnit 5, Mockito for testing
`spring-security-test`	Security testing utilities

4.2 Key Dependency Details

Spring Boot Starter Mail

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

Used for sending OTP emails via Gmail SMTP. Configured with `JavaMailSender`.

Hibernate (via Spring Data JPA)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Hibernate is the JPA implementation used to map Java entity classes to MySQL tables. `ddl-auto=update` automatically creates/updates tables on startup.

Lombok

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.36</version>
    <optional>true</optional>
</dependency>
```

Used on all entity classes with `@Data` to auto-generate getters, setters, constructors, and `toString`.

*Note: EduTrack does **not** use JWT tokens. Authentication is handled via custom BCrypt password validation in `AuthService`.*

5. OTP-Based Authentication

EduTrack implements OTP-based password reset for all three user roles (Student, Teacher, Admin).

5a. Security Configuration File

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())           // Disable CSRF for
REST APIs
            .cors(cors ->
cors.configurationSource(corsConfigurationSource()))
            .authorizeHttpRequests(auth -> auth
                .anyRequest().permitAll()           // All endpoints open
(auth handled manually)
            );
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of("*"));  // Allow all
origins
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);
        // ...
    }
}
```

5b. OTP Flow

```
User enters email + role
        ↓
OtpController validates email exists in correct role's table
        ↓
```

```
OtpService generates 6-digit OTP using SecureRandom
        ↓
Old OTPs for that email are deleted
        ↓
New OTP saved to otp_verification table (expires in 10 minutes)
        ↓
EmailService sends OTP via Gmail SMTP
        ↓
User enters OTP on frontend
        ↓
OtpService validates: not expired, not used, matches stored OTP
        ↓
OTP marked as used=true (single-use)
        ↓
AuthService resets password with BCrypt encoding
```

5c. OTP Security Rules

RULE	IMPLEMENTATION
Expiry	10 minutes from generation
Single-use	`used=true` after first successful validation
Role validation	Email checked against correct table before sending
Secure generation	`SecureRandom.nextInt(900000) + 100000`
Old OTP cleanup	`otpRepo.deleteByEmail(email)` before creating new

6. Password Security

All passwords are hashed using **BCrypt** before storing in the database.

```
private final BCryptPasswordEncoder encoder = new
BCryptPasswordEncoder();

// Encoding
student.setPassword(encoder.encode(rawPassword));

// Verification
encoder.matches(rawPassword, storedHash)
```

BCrypt automatically handles salting — each hash is unique even for the same password.

7. Entity Classes

All entity classes use **Lombok** annotations and **JPA** annotations:

```
@Entity
@Table(name = "students")
@Data                    // Lombok: generates getters, setters, toString
@NoArgsConstructor       // Lombok: no-args constructor
@AllArgsConstructor      // Lombok: all-args constructor
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer studentId;

    @Column(nullable = false, unique = true)
    private String email;

    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    private String password;   // Never returned in API responses

    // ... other fields
}
```

8. Repository Layer

Repositories extend `JpaRepository` which provides built-in CRUD operations:

```
@Repository
public interface StudentRepository extends JpaRepository<Student,
Integer> {
    Optional<Student> findByEmail(String email);
    boolean existsByEmail(String email);
    Optional<Student> findByUsn(String usn);
    List<Student> findByDepartmentIgnoreCase(String department);
}
```

Custom query methods are derived from method names — Spring Data JPA automatically generates the SQL.

9. Admin Auto-Seeding

On first startup, a default admin account is automatically created using `AdminSeeder.java`:

```
@Component
public class AdminSeeder implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) {
        if (!adminRepo.existsByEmail("admin@edutrack.com")) {
            Admin admin = new Admin();
            admin.setEmail("admin@edutrack.com");
            admin.setPassword(encoder.encode("Admin@123"));
            adminRepo.save(admin);
        }
    }
}
```

Default credentials:
Email: `admin@edutrack.com`
Password: `Admin@123`

10. Bulk Import (CSV)

The `BulkImportController` handles CSV file uploads for:

- Students (`/api/bulk/students`)
- Teachers (`/api/bulk/teachers`)
- Attendance (`/api/bulk/attendance`)
- Marks (`/api/bulk/marks`)

Uses **Apache Commons CSV** library for parsing:

```
<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-csv</artifactId>
    <version>1.9.0</version>
</dependency>
```